

Seed City

A Minecraft mod where a working redstone city grows itself from a seed, is built and maintained by mobs, and can be steered by the player.

Design document, v0.3 - September 2026

Part I: the city that grows. Part II: the program behind it. Part III: implementation reference.

Read this first. This document is the complete specification for a Minecraft mod called Seed City and is written to be handed to an implementing agent with no other context. Parts I and II describe what the mod is and why. Part III is the build reference: repo layout, data formats, interfaces, phase acceptance tests, and conventions. Where Parts I-III disagree, Part III wins. Where this document is silent, choose the simplest option that keeps every phase independently shippable, and record the decision in DECISIONS.md.

Part I - The city that grows

1. The one-sentence version

You place a single block. Builder mobs emerge from it and begin constructing a city outward, block by block, using a grammar of verified redstone cells. The city is a machine: at its core is a place to feed it a program, its lights and doors and elevators are that program's output, and its mobs are its organs. Give it no program and it runs a default; give it code and the builders construct whatever hardware the code needs. It keeps growing until it is contained. Every part of it actually works.

The core idea that makes everything else clean: **the builder mobs are the world generator**. There is no separate worldgen pass and separate expansion system and separate player tool. There is one mechanism - a builder holding a blueprint - and the seed, the spread, and player control are all just that mechanism at different scales.

2. Why this is worth building

Nothing like this exists. The pieces do: Verilog-to-redstone compilers, city generators like Lost Cities, circuit-compacting mods like Short Circuit. Nobody has fused them into a self-growing, functional, inhabited machine. A first release with a handful of cell types and one strange mob would already be a distinct thing on Modrinth.

It is also the same instinct as TreeOS pointed at a game: compiled logic with a spatial, inspectable output, grown from a root by rules. The project is a systems-architecture problem wearing a Minecraft costume.

3. Design pillars

- **Everything works.** No decorative redstone. A cell only enters the library if it has been verified in a headless simulator. Broken machines never spawn.
- **Show effects, not circuits.** The player is never asked to admire 400 repeaters. They see lights pulse in sequence, a drawbridge lower on a cycle, a mob wake when a bus goes high. The circuit is the substrate.
- **Random but legal.** The generator is a grammar, not noise. Randomness chooses *which* valid machine you get; the grammar guarantees it runs.
- **Set piece first, circuit second.** Every cell must be interesting to stand next to. If it is not, it does not ship.
- **One mechanism.** Builders with blueprints. The seed is a builder. Expansion is builders. Player commands are builders.

4. The seed and the spread

4.1 The Seed block

A single craftable block. When placed and powered it becomes the city's root: it holds the world seed offset, the grammar parameters (density, height, aggression, palette), and spawns the first Builder. Everything the city becomes is deterministic from the seed block's position plus the world seed, so two players placing the same seed in the same spot get the same city.

4.2 Growth as a frontier

The city has a frontier: the set of unbuilt cells adjacent to built ones. Builders live on the frontier. Each builder picks an open slot, asks the grammar for a legal cell that fits the neighbours' ports, and constructs it over time - visibly, block by block, with sound. Growth is therefore *slow, watchable, and physical*. You can stand at the edge and watch the city arrive.

4.3 The virus framing

Left alone, the city keeps growing. That is the hook and also the danger, so containment is a real gameplay layer:

- Builders need material. They pull from the city's own storage cells; when supply is low, growth stalls. Cutting supply lines starves the frontier.
- Builders will not cross certain blocks (a configurable warding block), so players can fence the spread.
- Killing the Seed block does not delete the city, but freezes all growth and repair. The city becomes a ruin that still runs whatever state it was in.
- Server config: max radius, max builder count, and a hard chunk cap so a forgotten seed cannot eat a world.

5. Cells and the grammar

5.1 The cell library

A cell is a hand-built, verified redstone module saved as structure NBT, with a fixed footprint and declared ports (signal in/out on specific faces at specific positions). Cells are the only thing the generator knows how to place. Everything the city can be is a composition of cells.

Cell	Function	What the player sees
Clock Tower	Master clock, divides into sub-clocks	A tall pulsing spire; the city's heartbeat
Bus Segment	Carries N-bit signal along a street	A street with glowing channels under glass
Register Block	Stores state	A vault-like building with lit indicator windows
Decoder Plaza	Selects one of many outputs	A plaza where one of several doors opens per cycle
Elevator Shaft	Piston lift driven by counter	A working lift that stops on floors by schedule
Drawbridge	Output actuator on a bus bit	A bridge that rises and falls with the program
Storage Cell	Material supply for builders	A warehouse the city draws from
Spawner Cell	Emits a mob when a state is reached	A shrine that wakes something when the lights align
Fault Cell	Deliberately broken segment	Dark district; a repair opportunity
Decor Cell	No ports; facade only	Cladding, lanterns, wood trim - what makes a district look lived in

Start with six to eight cells. The library is where the game's character lives and it is the part that must be hand-authored in-game. Claude Code writes the loader, the port schema, and the placement engine; the cells themselves are craft.

5.2 The grammar

Generation works like a dungeon generator with ports. Given an open slot and its neighbours' declared ports, the grammar returns the set of cells whose ports are compatible, weighted by district rules (a residential district wants more Register Blocks and fewer Clock Towers, and so on). Streets are buses; districts are subsystems; the Clock Tower is the root. This is placement-and-routing, and it is what makes random and logical stop being in tension.

5.3 Verification

Before a cell is committed to the world it is instantiated in a headless redstone tick simulator (MCHPRS-style, or a minimal in-mod one) with its neighbours and stepped for a few hundred ticks. If any port disagrees with the cell's declared truth table, the placement is rejected and the grammar tries again. This is the guarantee behind "everything works," and it is the single most important piece of engineering in the project.

6. The mobs

Mobs are the city's organs and its interface. Each role is a readout of some part of the machine, and each is a place the player can grab hold of it.

Mob	Role	Behavior
Builder	Grows the city	Lives on the frontier. Holds a blueprint (a cell). Fetches material from Storage Cells, constructs the cell block by block, moves on. Spawned by the Seed and by Spawner Cells as the city grows.
Warden	Repairs its district	Assigned to a district on spawn. Patrols it. When a cell's ports fail verification (a player broke it, an explosion, a Fault Cell), the Warden walks to it and rebuilds from the blueprint. Kill or lure the Warden and the district stays broken.
Courier	Carries signal	Physically moves a signal between districts that are not wired together - a living wireless link. Watching couriers tells you where information flows.
Collector	Feeds the city	Leaves the frontier to mine redstone, stone, and wood from surrounding terrain and hauls it to the nearest Storage Cell. The city visibly consumes its surroundings. See section 20.
Sentinel	Guards on state	Dormant at signal 0, hostile at 15, and in between behaves proportionally. A living gauge of a bus. Sleeping sentinels around a vault tell you the machine is not in the unlock state yet.

6.1 Player control of builders

This feature is included because it simplifies the design rather than complicating it. A tamed or captured Builder accepts a blueprint from the player. Hand it a Register Block blueprint and stand where you want it; the Builder constructs a verified, working register there. The player gets specific redstone without hand-building it, using the exact same code path the city uses to grow. No second system.

Builders can also be given a blueprint the player authored: build a circuit on a marked breadboard area, verify it through the same simulator, and it becomes a new cell the Builder can reproduce. This is how the cell library grows past what ships in the mod, and it means players can teach the city new organs.

7. What the player actually experiences

The arc, in order:

- **Discovery.** A pulsing spire on the horizon. A street that glows. Doors opening in sequence for no visible reason. Curiosity is the first hook: *why does it do that?*
- **Reading the machine.** Sentinels asleep here, awake there. Couriers running a fixed loop. Indicator windows on a vault counting up. The city is legible if you look.

- **The stuck city.** Every generated city ships with faults - dark districts whose Warden is missing or whose supply is cut. Find the break, reroute the signal, restore power. Puzzles fall out of the fact that the thing is a real machine.
- **Loot behind logic.** Vaults open only when the machine reaches a state. Reverse-engineer the program and you get the reward. This is the mid-game.
- **Owning a seed.** Craft your own Seed, place it, and watch a city grow from your base. Fence it, feed it, steer it with blueprints. This is the endgame, and it is also the thing people will make videos of.

The anti-jank rule holds all of this together: the player is shown behavior, not circuitry. A huge city is fine as long as it is a huge *dungeon* - more rooms worth finding - not a huge pile of wire.

8. Terrain and blending

Phase one is a bounded structure: the **city boat**. A floating platform with its own Seed, a self-contained machine, and a hard footprint. No biome edge cases, no terrain collision, fast to make playable and easy to show.

Phase two treats the city as a biome. A surface rule lets redstone plazas fade into normal ground at the edges instead of hard-cutting; the frontier follows terrain contours and Builders will not build below water or above a slope limit. Terrain blending is real work and should not gate the first release.

9. Technical plan

Layer	Approach
Platform	Fabric, latest stable Minecraft. Java. Claude Code drives the repo end to end.
Cells	Structure NBT plus a JSON sidecar declaring footprint, ports, truth table, district weights, and visual tags.
Grammar	Port-compatibility solver over the frontier. Weighted random selection with district rules. Deterministic from seed.
Verification	Headless tick simulator. Options: embed a minimal redstone sim, or shell out to an MCHPRS-style engine during dev and cache verified cell-pair results at build time.
Builders	Custom entities with a task queue: fetch, path, place, verify, advance frontier. All construction goes through one BuildTask type.
Wardens	Periodic port check per district; on failure, enqueue a repair BuildTask against the stored blueprint.
Player control	A Blueprint item. Right-click a Builder to assign. Breadboard block plus verification pass to author new blueprints.
Collectors	Custom entity with a gather task: find nearest permitted resource block within radius, mine, return to Storage Cell. Governed by the same priority queue as Builders.

Layer	Approach
Priority	A single ordered district list held by the Core, recomputed each program change. Every BuildTask and gather task is dispatched in that order.
Containment	Config: max radius, max builders, max collectors, chunk cap. Warding block. Seed destruction freezes growth.

10. Phases

Phase	Deliverable	Done when
0	Cell format, loader, port schema, six hand-built cells	A cell can be placed by command and its ports verified
1	City boat: Seed block, Builder mob, frontier grammar, default program	Place a seed on a platform; watch a working machine grow to the edge and run its idle loop
2	Wardens, Fault Cells, Sentinels	Break something; watch it get repaired. A dark district you can fix.
3	The Core, punch cards, behavior scripts (L1), Reader cell	Feed the city a card; the bridge now obeys your rule instead of the default
4	Player blueprints, Couriers, vault logic, hardware compilation (L2), Forge district	Hand the city a netlist and watch it build an adder in the industrial zone
5	Terrain blending, biome, containment polish, self-authored programs (L3)	A seed on open ground grows a city that fits the land and, when idle, writes its own cards

Phase 1 is the milestone that proves the whole idea. Everything after it is expansion of one working mechanism.

11. Risks

- **Simulator fidelity.** If the headless sim diverges from real Minecraft tick behavior, verified cells can still fail in-world. Mitigation: verify cell pairs, not whole cities, and cache results; run an in-world audit pass when a cell is first placed.
- **Performance.** Many active redstone cells in loaded chunks are expensive. Mitigation: cells are clocked by the Clock Tower, so unloaded districts pause; keep cell tick footprints small; bus segments should be observer-light.
- **Visual sameness.** A grammar of eight cells will repeat. Mitigation: palette variation per district, and the cell library is designed to grow via player blueprints.
- **Scope.** This is a multi-week project even with Claude Code doing the typing. The phase gates exist so that a stop at any phase still yields a real, shippable thing.

Open questions for Tabor

- How big is the instruction set? Part II proposes about twelve ops. Smaller is better for a solo build; the temptation will be to grow it.
- Mob aesthetic: mechanical, organic, or something in between? This decides the whole visual identity.
- Should the Seed be findable in the world before it is craftable, so discovery comes before ownership?

Part II - The program behind the city

12. The problem Part I leaves open

A city of working elevators is still a city of useless elevators unless something is deciding when they move. Part I builds a machine. Part II gives it a mind: a single input point, a small language, and a rule that what the city builds is whatever the program requires. This is what makes the whole thing one system rather than a set-piece generator with mobs on top.

The reference image is a punch-card mainframe. You submit a job. The machine chews on it. The result comes back as state you can read. Seen from above, a CPU already looks like a city; this project just takes that literally and lets you walk the streets.

13. The Core

The Seed block does not stay a block. As the first cells go up around it, the Seed is enclosed into the **Core**: a chamber at the city's center, always the first cell placed, always reachable. It is the one place a program enters the city.

- **Input: the Card Reader.** A slot that accepts a Punch Card item. Cards are craftable and writable (a written-book-style interface under the hood, punch-card styling on top). Insert a card and the Core parses it.
- **Output: the Reader wall.** A bank of indicator lamps and a signed-text display showing register contents and the current program state. The city's console. This is also where vault conditions are visible if you know how to read them.
- **Priority.** The Core's district is the one Wardens defend first and Builders never starve. Cut the Core off and the city runs its last program until something breaks.

Everything about the Core reuses Part I. It is a cell in the library with special weight, its lamps are bus outputs, its Warden is a Warden. The only new mechanic is the card slot.

14. Four levels of program

The layering is the thing that keeps this buildable by one person. Each level is a contract with the level below it, and a level can ship alone.

Level	What the player supplies	What the city does	Ships in
L0 - Default	Nothing	Runs a built-in idle program: a counter, a light choreography, a bridge cycle, a slow breath. This is what Part I already describes.	Phase 1

Level	What the player supplies	What the city does	Ships in
L1 - Behavior	A card of rules over existing hardware, e.g. when register A exceeds 5, raise the bridge; every 40 ticks, open door 3	Rewires actuators to the program without building anything new. Sentinels and doors now obey the player's rule instead of the default.	Phase 3
L2 - Hardware	A card describing a circuit: a netlist of cells and connections, or a short HDL	Compiles it into a build queue. Builders construct the hardware; the grammar routes it. Small circuits go up near the Core; big ones go to the Forge district.	Phase 4
L3 - Self-authored	Nothing, again	When the card slot is empty and the frontier is idle, the city composes its own cards from a fragment library and runs them. The player finds these as cards in the Reader: the city's dreams.	Phase 5

L0 through L2 are the real product. L3 is a small amount of code on top of L1 (mutate a card, run it, keep it if it verifies) and exists because a city that occasionally does something you did not ask for is the difference between a machine and a place.

15. The language

15.1 Keep it tiny

The single biggest risk in Part II is designing a language. The defense is to make it embarrassingly small and refuse to grow it until Phase 4 is done. Proposed: a register machine with about twelve operations.

Group	Ops	Notes
Data	SET, ADD, SUB, AND, OR, NOT	Registers are 4-bit to match redstone signal strength 0-15. Registers are physical Register Block cells.
Flow	JMP, JZ, WAIT n	WAIT is the interesting one: it ties the program to the Clock Tower and makes everything visible at human speed.
World	OUT port, IN port	OUT drives an actuator (bridge, door, lift, lamp). IN reads a sensor cell (pressure plate plaza, daylight, a Sentinel's alertness).
Build	NEED cell	L2 only. Declares that the program requires a cell that does not yet exist. This is the line that makes builders build.

A card is a short list of these. The Core stores the program in Register Blocks and the Clock Tower steps it. Because registers are real cells, the program is physically present in the city: you can walk to the block that holds the loop counter.

15.2 From card to construction

The pipeline is the same for every level and is the spine of the whole mod:

- **Parse.** Card text to an op list. Reject with a readable error on the Reader wall; a bad card should never crash a city.
- **Resolve.** Every OUT, IN and NEED is matched to a cell. Existing cells are bound; missing ones become build requests.
- **Place.** Build requests go to the frontier grammar as goals. Small requests: nearest legal slot to the Core. Large requests: routed into the Forge district.
- **Build.** Builders pull the queue exactly as in Part I. Nothing new here, which is the point.
- **Verify.** Each placed cell passes the headless simulator; the program does not go live until every NEED is satisfied and verified.
- **Run.** The Clock Tower steps the program. The city is now doing what the card said.

16. The Forge district

Part I's aesthetic rule is that every cell must be a set piece. Compiled hardware breaks that rule: an adder is not pretty and a CPU is thousands of cells with tight timing. Rather than fight it, give it a zone.

The Forge is the city's industrial district: dense, dark, loud, visibly different. It is allowed to be ugly in the way a real city's port or refinery is. Inside it, the grammar relaxes its visual weights and tightens its routing: bus segments are short and straight, cells pack edge to edge, Couriers are replaced by direct wire. From a distance it reads as the chip-from-above image, which is exactly the feeling to chase.

Two practical rules keep the Forge from swallowing the project:

- **Forge cells are a separate library.** Compact, verified, no aesthetic requirement. This is where the Verilog-to-redstone compiler output lives; it never mixes with the set-piece cells.
- **Timing is local.** The Forge has its own clock divider off the Clock Tower. The city does not need to run at the Forge's speed and the Forge does not need to wait for a drawbridge.

17. How the mobs change

Nothing in Part I's mob design is replaced; the program just gives each role a reason.

- **Builders** build to the queue. With no program the queue is the grammar's random frontier. With a program it is what the program needs. Same mob, different source of orders.
- **Wardens** prioritize by program dependency. A district the current card reads from outranks one it does not.
- **Couriers** are the physical form of OUT and IN across districts. You can watch a value travel.

- **Sentinels** can be IN sources: a program can branch on whether the Sentinels are awake, which means a program can react to the player.

18. Where the punch-card computer ends and the stretch begins

The ladder from L0 to L2 is the project. Everything past it is a stretch goal, and it is worth naming where the line is so the stretch stays a stretch:

- **In scope:** counters, light choreography, conditional actuators, small adders and comparators, a vault that opens on a computed state, a program the player wrote running visibly in the streets.
- **Stretch:** a general-purpose CPU compiled from a card. Real, since people have hand-built redstone CPUs that run games, but place-and-route at that scale is a solver problem an order of magnitude past the frontier grammar. If it ever happens it happens entirely inside the Forge.
- **Not this project:** a full Minecraft-in-Minecraft. Fun to say, not a milestone.

19. Building this alone without drowning

The method is to think only at the level of contracts and let the details fill in underneath. There are five interfaces in this document, and every piece of code belongs to exactly one of them:

Interface	One side	Other side
Cell	Structure NBT + JSON ports	Anything that places blocks
Grammar	A frontier slot + neighbour ports (+ optional goal)	A legal cell choice
BuildTask	A cell + a position	A Builder that executes it
Verify	A cell + its neighbours	Pass / fail
Card	Text	An op list, then build requests

If a task cannot be described as work on one of those five, it is either a cell to hand-build in-game or it is out of scope. That is the filter. Claude Code owns the code for all five; Tabor owns the cell library, the Forge cell library, and the fragment library for L3. The phases in section 10 are ordered so that each one is a working game on its own.

20. Supply and rule

20.1 Collectors close the loop

Part I says Builders draw from Storage Cells and stall when they run dry, but nothing filled the storage. Collectors do. They are the only mob that routinely leaves the city. Each Collector holds a gather order (redstone, stone, or wood), walks to the nearest permitted source within the containment

radius, mines it, and returns it to the nearest Storage Cell. Consequences that fall out for free:

- The city visibly consumes its surroundings. Forest thins, hillsides get quarried, ore veins empty. The virus framing becomes literal.
- Starving the city is a strategy: guard the ore, fence the collectors, replant nothing.
- Growth rate is a function of the land. A seed on a mountain with exposed redstone grows fast; one on a plain crawls. That is an economy in a single mob.
- Collectors will not cross Warding blocks and will not mine player-placed blocks (tracked by a placed-by-player flag), so they cannot eat a base.

20.2 Decoration is a Builder with a decor blueprint

There is no decorator mob. A Decor Cell is a cell with zero ports: cladding, lanterns, trim, a facade that sits over glass bus channels or wraps a Register Block. The grammar places Decor Cells exactly as it places logic cells, using district weights. A residential district carries a high decor weight; the Forge carries none. Making the city look more normal is therefore a weighting decision, not a system, and the simplest Decor Cell is a single block placed repeatedly - which is the fallback if nothing fancier ships.

20.3 Rule flows down from the Core

The hierarchy is Seed, then Core, then districts, then cells. It is not a mob giving orders; it is a queue whose ordering is set at the top. The Core holds a single ordered list of districts, recomputed whenever the program changes:

- Districts the current card reads from or writes to come first, in program order.
- The Core's own district is always first.
- Districts with open Fault Cells come before healthy ones of the same rank.
- Everything else is ordered by distance from the Core, nearest first.

Every BuildTask and every gather task is dispatched in that order. Wardens repair in that order. When material is scarce, low-ranked districts stall first and the program-critical ones keep running. That is the whole of rulership, and it is about forty lines of code.

Part III - Implementation reference

This part is for the agent building the mod. It assumes Parts I and II have been read once.

21. Stack and repo layout

Item	Choice
Loader	Fabric, targeting the latest stable Minecraft release at project start. Record the exact version in gradle.properties and do not float it.
Language	Java 21. No Kotlin. Keep dependencies to Fabric API only unless a phase note says otherwise.
Mod id	seedcity
Structure	Single Gradle module. Packages under net.tabor.seedcity: cell, grammar, build, verify, card, entity, core, config.
Data	src/main/resources/data/seedcity/cells/ holds NBT + JSON pairs. cards/ holds built-in cards including the L0 default. fragments/ holds L3 fragments.
Docs	DECISIONS.md at repo root: every judgement call the agent makes, one line each, dated. README.md: how to build, how to add a cell.

22. Data formats

22.1 Cell sidecar JSON

Each cell is a structure NBT file plus a JSON sidecar of the same name. The sidecar is the only thing the grammar and the verifier read.

```
{
  "id": "seedcity:register_block",
  "kind": "logic", // logic | actuator | sensor | storage | decor | core
  "size": [7, 6, 7], // x, y, z footprint in blocks
  "ports": [
    { "name": "in", "dir": "in", "face": "north", "pos": [3, 1, 0], "bits": 4 },
    { "name": "out", "dir": "out", "face": "south", "pos": [3, 1, 6], "bits": 4 },
    { "name": "clk", "dir": "in", "face": "west", "pos": [0, 1, 3], "bits": 1 }
  ],
  "truth": "register", // named verifier model, see 22.3
  "weights": { "core": 3, "residential": 4, "forge": 6, "plaza": 1 },
  "cost": { "redstone": 24, "stone": 80, "wood": 0 },
  "setpiece": true // false only for forge and decor cells
}
```

Rules: port pos is relative to the cell origin (min corner). A port on a face must lie on that face. Two cells are compatible when an out port of one lands exactly on an in port of the other with equal bits when the cells are placed adjacent. Ports of bits 4 are carried as signal strength 0-15; bits 1 as on/off.

22.2 Punch Card format

A card is plain text, one op per line, comments after a semicolon. Registers are R0-R7 and are bound to physical Register Block cells at resolve time. Ports are named after the cell and port that own them.

```
; bridge opens when the counter passes 5
SET R0 0
loop:
ADD R0 1
JZ R0 loop ; wraps at 16, restart
WAIT 20
OUT bridge.raise R0
JMP loop
NEED adder_4bit ; L2 only: build this if absent
```

The complete op set is SET, ADD, SUB, AND, OR, NOT, JMP, JZ, WAIT, OUT, IN, NEED. Do not add ops before Phase 4 is accepted. Parse errors are reported on the Reader wall as a line number and never throw.

22.3 Verifier models

Each cell names a truth model. A model is a small Java class implementing `boolean check(PortState before, PortState after, int ticks)`. Ship these: passthrough, not, and, or, register, counter, decoder, actuator (output changes when input does), sensor (output is read-only), none (decor and storage). The verifier instantiates the cell and its bound neighbours in a headless world, drives every input combination the model requires (bits 1 exhaustively, bits 4 over 0, 1, 7, 15), steps the configured tick budget, and asks the model. Cache pass results keyed by (cell id, neighbour ids, orientation) so a verified pairing is never re-run.

23. The five interfaces, precisely

Interface	Signature (sketch)	Owner package
Cell	<code>Cell load(Identifier id); Box footprint(BlockPos origin, Rotation r); List<Port> ports(Rotation r)</code>	cell
Grammar	<code>Optional<Placement> choose(FrontierSlot slot, List<BoundPort> neighbours, Optional<Goal> goal, Random rng)</code>	grammar
BuildTask	<code>record BuildTask(Cell cell, BlockPos origin, Rotation r, Cost cost); boolean step(BuilderEntity b)</code>	build
Verify	<code>Result verify(Placement p, List<Placement> neighbours)</code>	verify

Interface	Signature (sketch)	Owner package
Card	Program parse(String text) throws CardError; List<Goal> resolve(Program p, CityState s)	card

The Core (package core) owns CityState: the seed origin, the district list and its priority order, the frontier set, the storage totals, the current Program and its register bindings. Entities (package entity) only ever consume tasks from CityState; they never mutate the grammar or the program. This is the rule that keeps the codebase small.

24. Entity behavior specs

Entity	Loop	Limits and edge cases
Builder	take next BuildTask in priority order -> path to Storage Cell with the cost -> withdraw -> path to origin -> place blocks at a fixed rate (default 4 blocks per second) with sound -> on completion call Verify -> mark cell live or roll back and re-queue	Never places over player-placed blocks. Abandons a task if unreachable for 30 s and re-queues it. Config max_builders default 6.
Collector	take next gather order -> find nearest permitted source within radius -> path -> mine -> return to nearest Storage Cell -> deposit	Permitted sources: redstone ore, stone, logs, configurable. Never mines player-placed blocks or across a Warding block. Config max_collectors default 4.
Warden	assigned a district on spawn -> patrol its cells -> every 100 ticks run Verify on each live cell -> on failure enqueue a repair BuildTask at that district's priority	One Warden per district; respawns from the Core after 5 minutes if killed. Repair uses stored blueprint, so a player edit is reverted.
Courier	bound to one (OUT, IN) pair across districts -> read source port -> walk -> write signal at destination for one tick	Signal latency is walk time; that is intended. If the path is blocked the value simply does not arrive.
Sentinel	read its bound port each tick -> set alertness 0-15 -> at 0 sleep, at 15 hostile, in between wander and stare	Also exposes alertness as a sensor port so cards can IN from it.

25. Phase acceptance tests

A phase is done when every line under it passes in a fresh world. Write these as Gametest where the framework allows and as a manual checklist otherwise.

Phase	Acceptance
0	/seedcity place <cell> places any shipped cell. /seedcity verify <cell> reports pass for all six. A deliberately corrupted NBT reports fail with a port name.

Phase	Acceptance
1	Placing a Seed on a 48x48 platform spawns a Builder, encloses the Seed into a Core within 2 minutes, and fills the platform with verified cells within 20 minutes at default rates. The L0 default card runs: the Clock Tower pulses and at least one actuator cycles. Same seed and position on two worlds yields identical layouts.
2	Breaking a bus segment darkens downstream cells; the district Warden repairs it within 3 minutes. A generated city contains at least one Fault Cell. A Sentinel bound to a bus sleeps at 0 and attacks at 15.
3	Inserting a valid L1 card changes an actuator's behavior within one Clock Tower cycle. A card with a syntax error shows the line number on the Reader wall and leaves the previous program running. Removing the card returns to L0.
4	A card containing NEED adder_4bit causes Builders to construct a verified adder in the Forge; the program goes live only after verification. A player can hand a captured Builder a Register Block blueprint and it is built where the player stands. A Courier carries a value between two districts. A vault opens only on a computed state.
5	A Seed on natural terrain grows a city whose edge cells fade into ground via the biome surface rule; Collectors visibly deplete a nearby forest. With an empty card slot for 10 minutes the city writes and runs an L3 card that appears in the Reader. Config chunk cap halts growth exactly at the cap.

26. Conventions and non-goals

- **Determinism.** All randomness comes from one Random seeded by (world seed, seed block pos). No entity uses world.random for generation decisions.
- **Never crash a city.** Bad cards, bad NBT, unreachable tasks, missing cells: log, report on the Reader wall, degrade. A city is a long-lived object.
- **No new mobs.** Five entity types. New behavior is a new blueprint, a new card, or a new weight - not a new mob.
- **No new ops.** Twelve until Phase 4 ships.
- **Tick budget.** Unloaded districts do not tick. Loaded city cells should cost less than 5 percent of a 20 TPS server budget at 200 cells; measure this at Phase 1 and again at Phase 4.
- **Non-goals.** Multiplayer city ownership, cross-dimension seeds, Forge-loader support, a general-purpose CPU compiler, Minecraft-in-Minecraft. Each may be a later project; none is this one.

27. Glossary

Term	Meaning
Seed	The craftable block that starts a city. Becomes enclosed as the Core.

Term	Meaning
Core	Central cell holding the Card Reader, Reader wall, program state, and district priority list.
Cell	A verified redstone module with declared ports. The only unit the generator places.
Port	A signal connection point on a cell face; 1-bit on/off or 4-bit signal strength.
Frontier	The set of unbuilt slots adjacent to built cells. Where Builders work.
Grammar	The port-compatibility solver with district weights that chooses which cell fills a slot.
District	A region of the city with its own weights, Warden, and priority rank.
Forge	The industrial district where compiled hardware lives; dense, no set-piece requirement.
Card	A text program inserted at the Core. Levels L0 default, L1 behavior, L2 hardware, L3 self-authored.
Goal	A build request produced by resolving a card's NEED, OUT, or IN against missing cells.
Warding block	A player-placed block Builders and Collectors will not cross.
Set piece	A cell that is interesting to stand next to. Required of every non-Forge, non-decor cell.